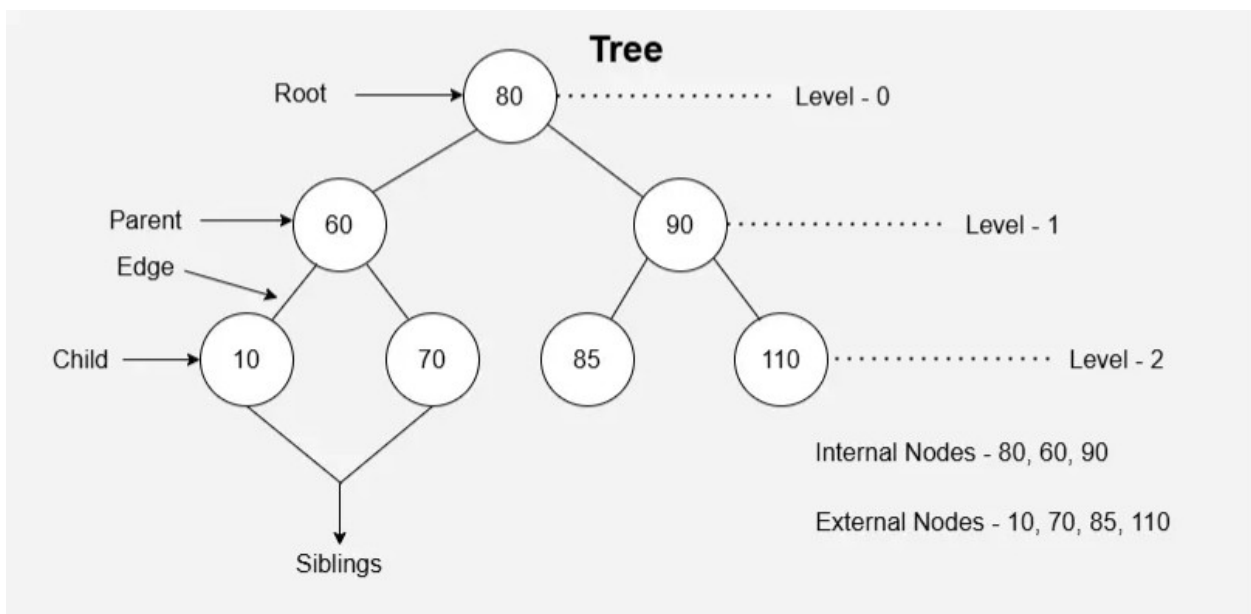
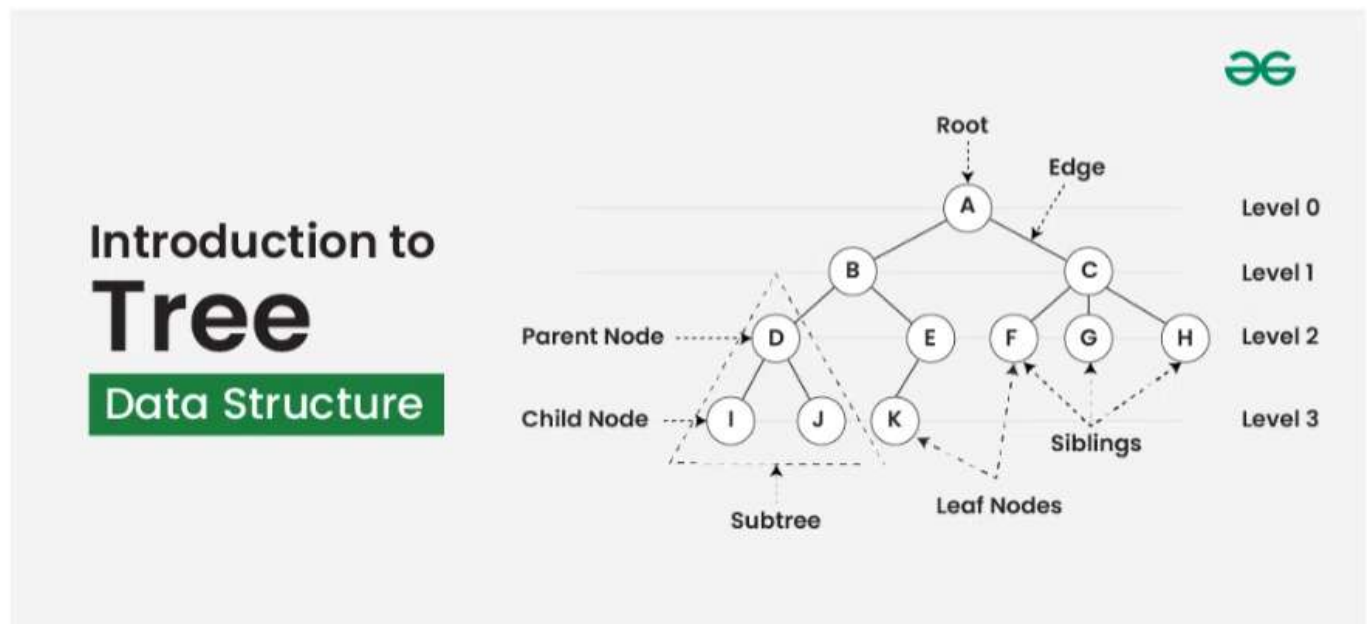


# Trees

Monday, May 18, 2026 10:21 AM



```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

## BINARY TREE

It is a non-linear and hierarchical data structure where each node has at most two children referred to as the left and right child.

**Number of edges:** An edge can be defined as the connection between two nodes. If a tree has  $N$  nodes then it will have  $(N-1)$  edges. There is only one path from each node to any other node of the tree.

**Depth of a node:** The depth of a node is defined as the length of the path from the root to that node. Each edge adds 1 unit of length to the path. So, it can also be defined as the number of edges in the path from the root of the tree to the node.

- **Height of a node:** The height of a node can be defined as the length of the longest

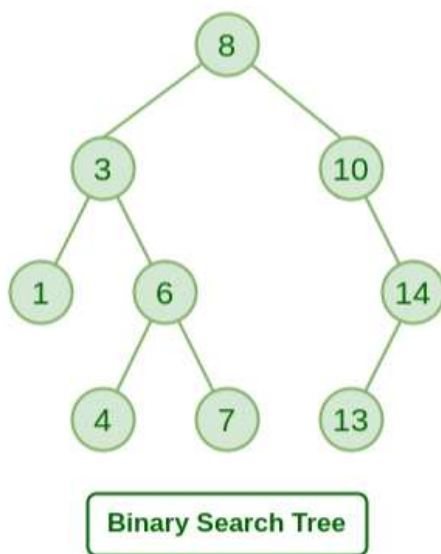
path from the node to a leaf node of the tree.

- **Height of the Tree:** The height of a tree is the length of the longest path from the root of the tree to a leaf node of the tree.
- **Degree of a Node:** The total count of subtrees attached to that node is called the degree of the node. The degree of a leaf node must be **0**. The degree of a tree is the maximum degree of a node among all the nodes in the tree.

## BINARY SEARCH TREE

A Binary Search Tree (or BST) is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing values less than the parent node and the right child containing values greater than the parent node. This hierarchical structure allows for efficient searching, insertion and deletion operations on the data stored in the tree.

The left side of a node only has smaller values. The right side of a node only has bigger values. Both left and right sides follow the same rule, forming a smaller BST.



*Binary Search Tree*

```
class Node:
    def __init__(self, key):
        self.value = key
        self.left = None
        self.right = None
```

```
# Creating the root node
root = Node(5)
```

### Insertion in BST

We need to traverse through all nodes until we find a leaf node and insert the node as the left or the right child based on the value of the leaf node.

```
# Python program to demonstrate
# insert operation in binary search tree
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
```

```
# A utility function to insert
# a new node with the given key
def insert(root, key):
    if root is None:
        return Node(key)
    if root.val == key:
        return root
    if root.val < key:
        root.right = insert(root.right, key)
    else:
        root.left = insert(root.left, key)
    return root
```

```
# A utility function to do inorder tree traversal
def inorder(root):
    if root:
        inorder(root.left)
        print(root.val, end=" ")
        inorder(root.right)
```

```
r = Node(15)
r = insert(r, 10)
r = insert(r, 18)
r = insert(r, 4)
r = insert(r, 11)
r = insert(r, 16)
r = insert(r, 20)
r = insert(r, 13)
```

```
# Print inorder traversal of the BST
inorder(r)
```

Time complexity: the worst case is  $O(h)$ .  
Space complexity:  $O(1)$

## Searching in BST

We don't need to traverse all nodes, it is based on the value contained in the nodes.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

# function to search a key in a BST
def search(root, key):
    # Base Cases: root is null or key
    # is present at root
    if root is None or root.key == key:
        return root

    # Key is greater than root's key
    if root.key < key:
        return search(root.right, key)

    # Key is smaller than root's key
    return search(root.left, key)
```

```
# Creating a hard coded tree for keeping
# the length of the code small. We need
# to make sure that BST properties are
# maintained if we try some other cases.
```

```
root = Node(50)
root.left = Node(30)
root.right = Node(70)
root.left.left = Node(20)
root.left.right = Node(40)
root.right.left = Node(60)
root.right.right = Node(80)
```

```
# Searching for keys in the BST
print("Found" if search(root, 19) else "Not Found")
print("Found" if search(root, 80) else "Not Found")
```

Time complexity:  $O(h)$

Space:  $O(h)$  for the recursion stack.

Deletion in BST

- 1) Deleting a node having both left child and right child. After finding the node, copy the value of the right child in  $v$  and copy the right child's left pointer to left of  $v$  and right pointer. We need to replace it with the maximum value of its left or right sub tree.
- 2) Deleting a node having either left or right child. We need to replace the value in the node with its left node if it has left child, or right node if it has right child.
- 3) Deleting a leaf

```
class Node:
```

```
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

```
# Note that it is not a generic inorder successor
# function. It mainly works when the right child
# is not empty, which is the case we need in BST
# delete.
```

```
def get_successor(curr):
    curr = curr.right
    while curr is not None and curr.left is not None:
        curr = curr.left
    return curr
```

```
# This function deletes a given key x from the
# given BST and returns the modified root of the
# BST (if it is modified).
```

```
def del_node(root, x):
```

```
    # Base case
    if root is None:
        return root
```

```
    # If key to be searched is in a subtree
    if root.key > x:
        root.left = del_node(root.left, x)
    elif root.key < x:
        root.right = del_node(root.right, x)
    else:
```

```
# If root matches with the given key
```

```
# Cases when root has 0 children or  
# only right child  
if root.left is None:  
    return root.right
```

```
# When root has only left child  
if root.right is None:  
    return root.left
```

```
# When both children are present  
succ = get_successor(root)  
root.key = succ.key  
root.right = del_node(root.right, succ.key)
```

```
return root
```

```
# Utility function to do inorder traversal
```

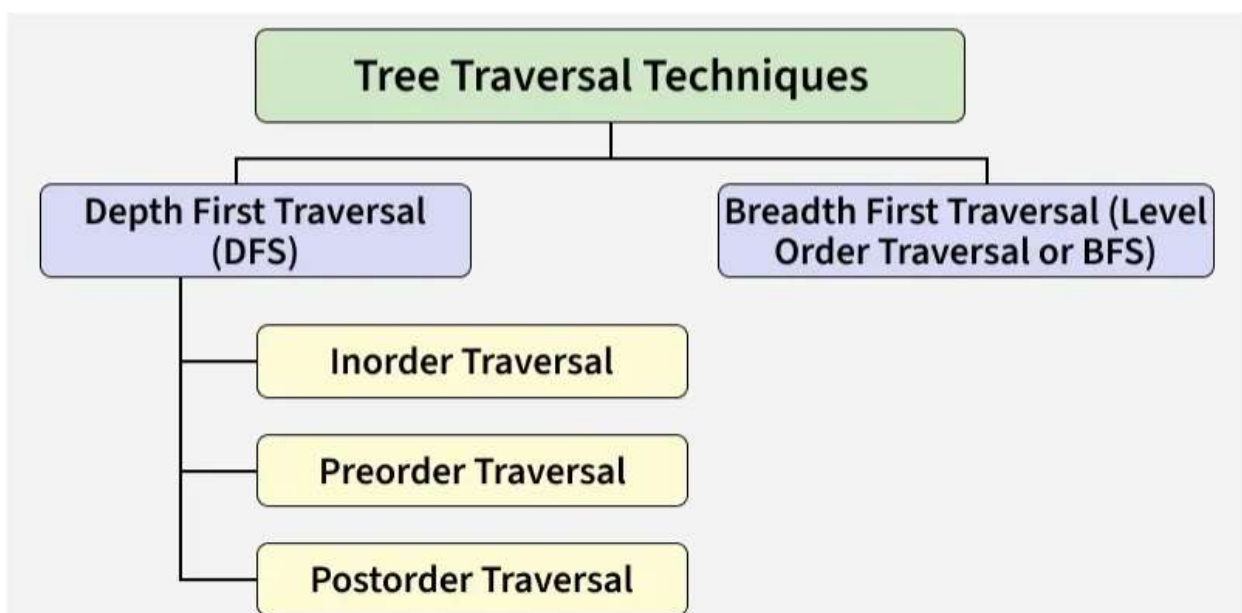
```
def inorder(root):  
    if root is not None:  
        inorder(root.left)  
        print(root.key, end=" ")  
        inorder(root.right)
```

```
# Driver code
```

```
if __name__ == "__main__":  
    root = Node(10)  
    root.left = Node(5)  
    root.right = Node(15)  
    root.right.left = Node(12)  
    root.right.right = Node(18)  
    x = 15
```

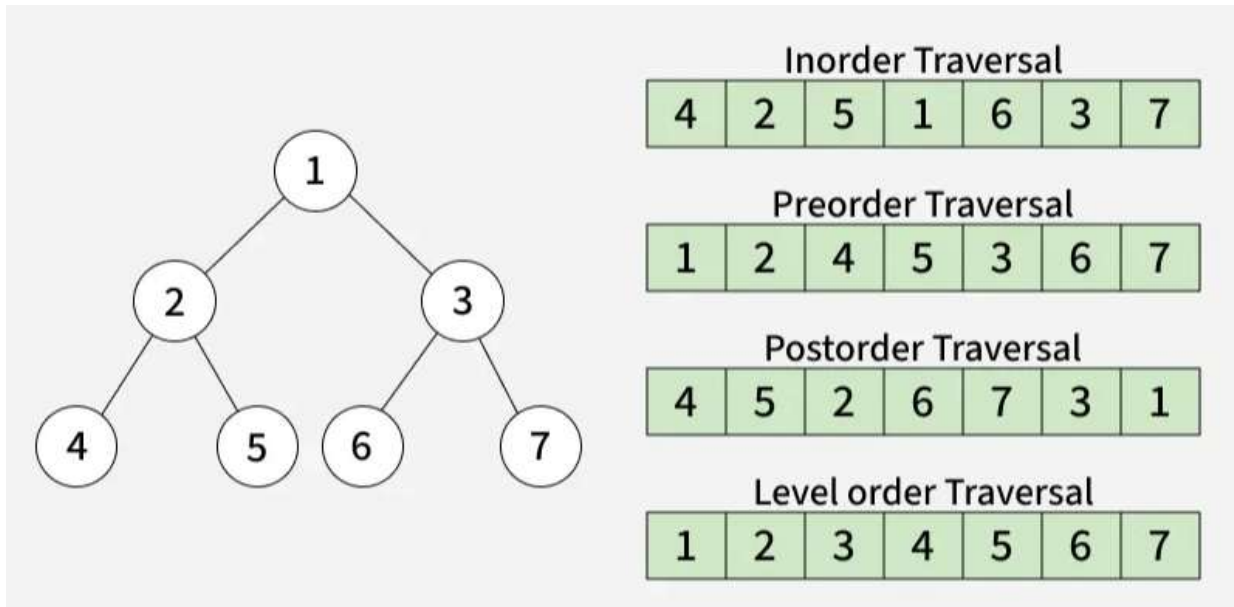
```
root = del_node(root, x)  
inorder(root)  
print()
```

## Traversals in BST



DFS: explores as far as possible along a branch before exploring the next branch.

BFS: explores nodes level by level from top to bottom.



**INORDER:** left -> root -> right

class Node:

```
def __init__(self, key):
    self.left = None
    self.right = None
    self.val = key
```

# A function to do inorder tree traversal

def printInorder(root):

if root:

# First recur on left child  
 printInorder(root.left)

# then print the data of node  
 print(root.val),

# now recur on right child  
 printInorder(root.right)

```
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
root.right.right = Node(6)
```

printInorder(root)

**PREORDER:** root -> left-> right

class Node:

```
def __init__(self, key):
    self.left = None
    self.right = None
    self.val = key
```

# A function to do preorder tree traversal

```
def printPreorder(root):

    if root:

        # First print the data of node
        print(root.val),

        # Then recur on left child
        printPreorder(root.left)

        # Finally recur on right child
        printPreorder(root.right)
```

```
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
root.left.right = Node(6)
printPreorder(root)
```

**POSTORDER:** left -> right -> root  
 # Python3 program to for tree traversals

# A class that represents an individual node in a  
 # Binary Tree

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
```

# A function to do postorder tree traversal  
 def printPostorder(root):

```
    if root:

        # First recur on left child
        printPostorder(root.left)

        # the recur on right child
        printPostorder(root.right)

        # now print the data of node
        print(root.val),
```

```
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
root.right.right = Node(6)

printPostorder(root)
```

Time complexity:  $O(N)$   
Space:  $O(\log N)$

## Level Order Traversal

- 1) Recursion:  $O(n)$  time and  $O(n)$  space. The idea is to traverse the tree recursively, starting from the root. When a node is visited, its values is added to the result array at the index corresponding to its level, and then its left and right children are recursively processed in the same way.

class Node:

```
def __init__(self, value):  
    self.data = value  
    self.left = None  
    self.right = None
```

```
def levelOrderRec(root, level, res):
```

```
    # Base case  
    if root is None:  
        return
```

```
    # Add a new level to the result if needed  
    if len(res) <= level:  
        res.append([])
```

```
    # Add current node's data to its corresponding level  
    res[level].append(root.data)
```

```
    # Recur for left and right children  
    level_order_rec(root.left, level + 1, res)  
    level_order_rec(root.right, level + 1, res)
```

```
# Function to perform level order traversal
```

```
def levelOrder(root):  
    # Stores the result level by level  
    res = []  
    level_order_rec(root, 0, res)  
    return res
```

```
if __name__ == '__main__':
```

```
    #      5  
    #    / \  
    #  12  13  
    # /  \  \  
    # 7   14 2  
    # /  \ /  \ / \  
    #17 23 27 3 8 11
```

```
    root = Node(5)  
    root.left = Node(12)  
    root.right = Node(13)
```

```
    root.left.left = Node(7)  
    root.left.right = Node(14)
```

```
    root.right.right = Node(2)
```

```
    root.left.left.left = Node(17)  
    root.left.left.right = Node(23)
```

```
    root.left.right.left = Node(27)
```

```
root.left.right.right = Node(3)
```

```
root.right.right.left = Node(8)
root.right.right.right = Node(11)
```

```
res = level_order(root)
```

```
for level in res:
    print(' '.join(map(str, level)))
```

2) Using queue:  $O(n)$  time,  $O(n)$  space

- Create an empty queue
- Enqueue the root node of the tree to Q
- Loop while Q is not empty:
  - o Dequeue a node from Q and visit it
  - o Enqueue the left child of the dequeued node if exists
  - o Enqueue the right child of the dequeued node if exists.

class Node:

```
def __init__(self, value):
    self.data = value
    self.left = None
    self.right = None
```

# Iterative method to perform level order traversal

```
def levelOrder(root):
    if root is None:
        return []
```

# Create an empty queue for level order traversal

```
q = []
res = []
```

# Enqueue Root

```
q.append(root)
curr_level = 0
```

while q:

```
    len_q = len(q)
    res.append([])
```

```
    for _ in range(len_q):
```

# Add front of queue and remove it from queue

```
    node = q.pop(0)
    res[curr_level].append(node.data)
```

# Enqueue left child

```
    if node.left is not None:
        q.append(node.left)
```

# Enqueue right child

```
    if node.right is not None:
        q.append(node.right)
```

```
    curr_level += 1
```

```
    return res
```

```
if __name__ == '__main__':
```

```
    # 5
    # / \
    # 12 13
```

```
# / \ \
# 7  14 2
# / \ / \ / \
#17 23 2 3 8 11
```

```
root = Node(5)
root.left = Node(12)
root.right = Node(13)
```

```
root.left.left = Node(7)
root.left.right = Node(14)
```

```
root.right.right = Node(2)
```

```
root.left.left.left = Node(17)
root.left.left.right = Node(23)
```

```
root.left.right.left = Node(27)
root.left.right.right = Node(3)
```

```
root.right.right.left = Node(8)
root.right.right.right = Node(11)
```

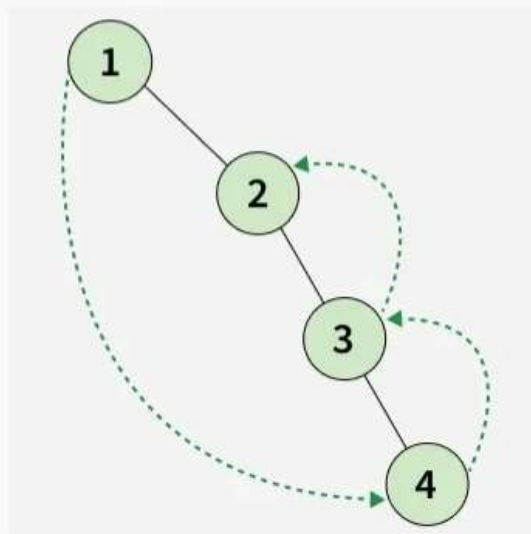
```
# Perform level order traversal and get the result
res = levelOrder(root)
for level in res:
    for val in level:
        print(val, end=' ')
    print()
```

## Boundary Traversal

It includes left boundary (nodes on left excluding leaves), leaves, right boundary (nodes on the right excluding leaves).

**Output :** [1, 4, 3, 2]

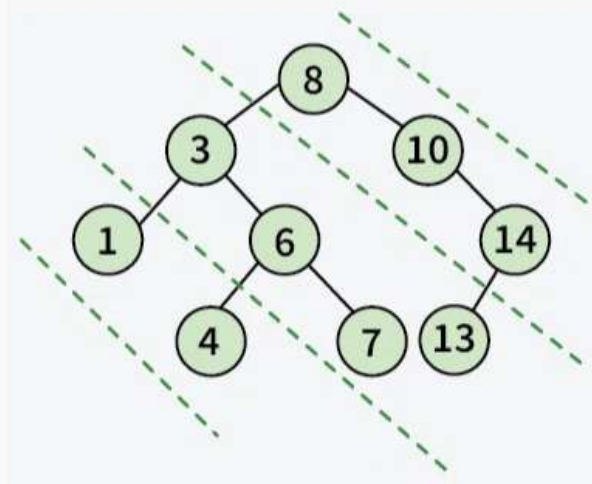
**Explanation:**



## Diagonal Traversal

All the nodes in a single diagonal will be printed one by one.

*Input:*



*Output:* 8 10 14 3 6 7 13 1 4

*Explanation:* The above is the diagonal elements in a binary tree that belong to the same line.

### Difference Between BFS and DFS:

| Parameters            | BFS                                                                                                                        | DFS                                                                                                                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stands for            | BFS stands for Breadth First Search.                                                                                       | DFS stands for Depth First Search.                                                                                                                                                       |
| Data Structure        | BFS(Breadth First Search) uses Queue data structure for finding the shortest path.                                         | DFS(Depth First Search) uses Stack data structure.                                                                                                                                       |
| Definition            | BFS is a traversal approach in which we first walk through all nodes on the same level before moving on to the next level. | DFS is also a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes. |
| Conceptual Difference | BFS builds the tree level by level.                                                                                        | DFS builds the tree sub-tree by sub-tree.                                                                                                                                                |
| Approach used         | It works on the concept of <b>FIFO (First In First Out)</b> .                                                              | It works on the concept of <b>LIFO (Last In First Out)</b> .                                                                                                                             |
| Suitable for          | BFS is more suitable for searching vertices closer to the given source.                                                    | DFS is more suitable when there are solutions away from source.                                                                                                                          |
| Applications          | BFS is used in various applications such as bipartite graphs, shortest paths, etc.                                         | DFS is used in various applications such as acyclic graphs and finding strongly connected components etc.                                                                                |